

Should AI Optimize Your Code? A Comparative Study of Classical Optimizing Compilers Versus Current Large Language Models

Miguel Angel Romero Rosas*
University of Delaware
Newark, Delaware, USA
miguelro@udel.edu

Miguel Angel Torres Sanchez*
University of Delaware
Newark, Delaware, USA
mgtorsa@udel.edu

Rudolf Eigenmann
University of Delaware
Newark, Delaware, USA
eigenman@udel.edu

Abstract

Traditional optimizing compilers have played an important role in adapting to the growing complexity of modern software systems. The need for efficient parallel programming in current architectures requires strong optimization techniques. The beginning of Large Language Models (LLMs) raises intriguing questions about the potential of these AI approaches to revolutionize code optimization methodologies. This work aims to answer an essential question for the compiler community: "Can AI-driven models revolutionize the way we approach code optimization?"

To address this question, we present a comparative analysis between three classical optimizing compilers and two recent large language models, evaluating their respective abilities and limitations in optimizing code for maximum efficiency. In addition, we introduce a benchmark suite of challenging optimization patterns and an automatic mechanism for evaluating the performance and correctness of the code generated by LLMs. We used three different prompting strategies to evaluate the performance of the LLMs – Simple Instruction (IP), Detailed Instruction Prompting (DIP), and Chain of Thought (CoT).

A key finding is that while LLMs have the potential to outperform current optimizing compilers, they often generate incorrect code on large code sizes, calling for automated verification methods. In addition, expressing a compiler strategy as part of the LLM's prompt substantially improves its overall performance. Our evaluation across three benchmark suites shows CodeLlama-70B as the superior LLM, capable of achieving speedups of up to x1.75. Additionally, CETUS is the best among the current optimizing compilers, achieving a maximum speedup of 1.67x. We also found substantial differences among the three prompting strategies.

CCS Concepts

• **Computing methodologies** → **Parallel programming languages**; **Artificial intelligence**; • **Software and its engineering** → **Software verification and validation**.

Keywords

Automatic parallelization, Compilers, Large Language Models

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution International 4.0 License.

SCA '25, Singapore, Singapore

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1250-0/25/03

<https://doi.org/10.1145/3718350.3718357>

ACM Reference Format:

Miguel Angel Romero Rosas, Miguel Angel Torres Sanchez, and Rudolf Eigenmann. 2025. Should AI Optimize Your Code? A Comparative Study of Classical Optimizing Compilers Versus Current Large Language Models. In *SupercomputingAsia 2025 (SCA '25)*, March 10–13, 2025, Singapore, Singapore. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3718350.3718357>

1 Introduction

The need for developing and optimizing parallel programs is more crucial than ever [18]. Optimizing compilers have long been one of the pillars of creating parallel code, evolving continuously to meet the growing complexity of modern programs and architectures. However, despite all advancements, current compilers still do not achieve the necessary performance to be true alternatives to manual parallelization [8].

The dawn of Large Language Models (LLMs), such as GPT and CodeLlama raises a fundamental question: **Can AI-driven models revolutionize the way we approach code optimization?** These models paint an appealing horizon for code optimization and parallel code generation [32]. Among several LLMs, we chose to explore GPT4.0 from OpenAI [30] and CodeLlama-70B [23] from META, as these represent the state-of-the-art and thus offer the promise of efficient code generation [22].

Our study compares three state-of-the-art optimizing compilers, CETUS [4], PLUTO [10] and ROSE [34] against the two LLMs for parallel code optimization. We evaluate their capacity to serve as Automatic Optimization Tools (AOTs).

One of the challenges in using LLMs for code generation is the lack of correctness guarantee. To address this critical gap, we developed a mechanism for automatic validation of the generated programs. The mechanism enables LLMs to be used as AOTs and facilitates the comprehensive evaluation of these models. The mechanism can also serve as a general instrument facilitating the use of unsafe optimizations. To the best of our knowledge this is the first such mechanism, enabling the automatic use of optimizations that do not ensure correctness.

In our evaluation, we are specifically interested in code patterns that challenge current automatic optimization tools and in the ability of LLM-based AOTs to overcome these challenges. To this end, we have developed a benchmark suite of such patterns. Next to a comprehensive evaluation, the suite is also available to identify areas for improvement in parallel code optimization.

This paper makes the following contributions:

- We present a comparative study of LLMs and traditional optimizing compilers, utilizing study cases from real-world applications, such as the NAS Parallel Benchmarks Suite

(NPB) v3.3 [5], the POLYBENCHMARK (PB) Suite v4.2 [38], and other relevant programs.

- We introduce an automated mechanism for validating the correctness and performance of AOT-generated code called PCAOT (Performance and Correctness Evaluation of Automatic Optimization Tools).
- We present a Parallel Computing Challenge Benchmark suite (PCB) v1.0 [26], comprising 28 parallel and non-parallel use cases for evaluating the capabilities of AOTs.

This paper is organized as follows: Section 2 explores strengths and limitations of our chosen optimizing compilers and LLMs. Section 3 describes our environment for evaluating automatic optimization tools and presents the Parallel Computing Challenge Benchmark suite (PCB). Section 4 evaluates the capabilities of our LLMs and the optimizing compilers on challenging code patterns for both parallel and non-parallel scenarios, followed by the discussion of related work in Section 5 and conclusions in Section 6.

2 LLMs vs Traditional Compilers, Strengths and Limitations

Current optimizing compilers exhibit a mix of challenges and opportunities [19, 28, 39]. Chief among them are limitations of static analysis, hampering the identification of optimization opportunities, and the complexity of deciding on best optimization patterns for maximum program performance [7]. It is in this context that the power of LLMs emerges, offering new potential ways to approach program optimization.

Large Language Models have captured significant attention in recent years owing to their statistical understanding of programs, offering suggestions for code generation, and facilitating software creation [20], [21], [1], [13]. While LLM-generated software has demonstrated success in many applications, recent studies have highlighted the limitations in capturing crucial aspects of linguistic meaning and understanding semantic properties of the user's prompt [2].

Regardless of these limitations, recent studies suggest that Large Language Models perform remarkably well in different programming tasks, such as code generation, bug fixing, and code completion [36]. This expertise is largely attributed to the amount of data available on platforms such as GitHub, which provides an effective training. However, even though LLMs can generate effective code, the lack of correctness guarantees make them unsuitable for many code optimization approaches [12, 17, 29, 35].

3 An Environment for Evaluating Automatic Optimization Tools

Recall that the lack of methods for verifying the correctness of code generated by LLMs makes it difficult to compare such tools with other AOTs. Section 3.1 introduces a novel method and environment called PCAOT (Performance and Correctness Evaluation of Automatic Optimization Tools) that overcomes this limitation. Additionally, we have created the Parallel Challenge Patterns Benchmark (PCB) suite v1.0 with 28 parallel and non-parallel use cases. The suite and the challenge patterns are discussed in Section 3.2.

3.1 Evaluation and Verification

Our approach entails selecting two Large Language Models (LLMs), GPT-4.0 and CodeLlama-70B, and evaluating their code optimization capabilities. The results are then compared with three different optimizing compilers, CETUS, PLUTO, and ROSE. For engaging the LLMs, we use three prompting strategies, described next.

3.1.1 Prompting Strategies. We explored three different prompts associated with utilizing Large Language Models for High-Performance Computing optimization tasks. Initially, we employed a simple instruction prompt (IP) [27] [16]. Additionally, we transformed the original simple instruction prompt into a more detailed one (DIP). Finally, we employed a Chain-of-Thought prompt (CoT). Recent studies indicate that Chain-of-Thought prompting enhances performance across a spectrum of arithmetic, commonsense, and symbolic reasoning tasks, making it well suited for assessing domain-specific knowledge [37].

CETUS, PLUTO, and ROSE translate C programs to equivalent C code annotated with OpenMP parallel directives. Incorporating OpenMP into the prompts facilitates the comparison of the code generated by LLMs and these compilers. We chose the following prompts:

- Simple Instructing Prompting (IP) - *"Given the program below, improve its performance using OpenMP."*
- Detailed Instructing Prompting (DIP) - *"Given the C program below, check for read after write and write after read dependencies among iterations, if there are no dependencies among iterations of the outermost loop, parallelize this loop using OpenMP directives. If dependencies are found in the outermost loop but there exist inner loops that can be parallelized without violating data dependencies, then parallelize those inner loops instead."*
- Chain-of-Thought Prompt - *"<DIP> + As you work through the program, explain each step of your reasoning process to ensure clarity and correctness in your optimization decisions. Think step by step."*

The main difference between the three types of prompts recline in their ability to guide the model's reasoning process. Instructing prompts provide a more direct and clear directive, resulting in the immediate production of an optimized program version. As opposed to Chain of Thought (CoT) prompts, which enable the model to break down the task into multiple steps, allowing for iterative refinement before generating an optimized program version. Instances of guidance in the model in this process are: **"Explain each step of your reasoning process..."**, **"Break down your thought process into individual steps..."**, **"Detail the reasoning behind your answer..."** [35].

We chose to add a condition into the prompts DIP and CoT which is **"check for read after write, and write after read dependencies among iterations, If there are no dependencies among iterations of the outermost loop, parallelize this loop using OpenMP directives..."**. This guidance consists of two key components: first, analyzing data dependencies to ensure correctness, and second, prioritizing the parallelization of outer loops. These key components represent two of the most fundamental concepts

in automatic program parallelization. Additionally, when prompted simply to **"improve its performance"**, with IP, LLMs are likely to attempt parallelization regardless of whether the code is actually parallelizable. The DIP and CoT prompts express high level compiler strategies that guide the LLMs to improve their reasoning process.

For the optimizing compilers, we used the default set of parameters, representing the known, safe optimization methods. Note that, while experimenting with optimization options could potentially yield improved performance, doing so is beyond the scope of this study. Our environment for evaluating the five AOTs comprises three phases: **Preparation**, **Optimization**, and **Validation**. Figure 1 illustrates this architecture.

3.1.2 Preparation Phase. LLMs have shown to have limitations in processing programs with large code sizes [31] [24]. Hence, our LLM evaluation focuses on section-level optimizations. The preparation phase identifies the sections within the source code to experiment with. Each such section is manually enclosed with **#pragma experimental section start** and **#pragma experimental section stop**, as illustrated in Figure 2. The impact of code size on the performance of the LLMs will be discussed in Section 4.2.

```
1: #pragma experimental start
2: for (k=0; k<=(grid_points[2]-1); k++)
3: {
4:     ...computation
5: }
6: #pragma experimental end
```

Figure 2: Experimental section instrumentation. Example taken from subroutine PO_Function_Version_1 of the PCB suite.

For our experiments, we made use of a tool called CaRV (Capture, Replay, and Validate), which enables users to experiment with sections of large applications. It facilitates the comparison of individual program sections before and after optimizations, assessing their efficiency and accuracy [6]. CaRV does so using a selective checkpointing method, based on the identified program states at the beginning and end of an experimental section. The preparation phase generates a new program version that is instrumented in a way that allows the optimized program section to be run independently and compared against the captured end state of the original program. Both the original and instrumented program versions are passed on to the optimization phase.

3.1.3 Optimization Phase. This phase sends the experimental section with the prompts to the two LLMs to obtain the optimized versions. The resulting code is then inserted into the instrumented program. The original code is also sent to the three compilers, CETUS, PLUTO, and ROSE. This process generates a total of 22 program versions, for the five AOTs (two LLMs, three different prompts, three requests per prompt, three optimizing compilers plus the serial program). The next phase executes and validates these versions.

3.1.4 Validation Phase. The Validation phase executes each version of the experimental section individually. The LLM-generated versions are then compared with the original, serial program versions, using the CaRV checkpoints. This process verifies the values

of all relevant, "live" variables at the end of the LLM-optimized program section against the output state of the corresponding values in the original program. The execution time is measured as well and compared to that of the original program section.

3.2 Parallel Computing Challenge Pattern Benchmark Suite v1.0 (PCB)

The PCB Suite was specifically created to address a critical need for unbiased evaluation by ensuring that it has not been exposed to LLMs during training. This new Benchmark suite comprises 28 distinct use cases, each presenting one of six challenging scenarios encountered by optimizing compilers [7]. A number of patterns are included in the PCB suite, testing the optimizers in their ability to: **parallelize outermost loops (PO)**, **parallelize loops with function calls (PF)**, **form parallel regions enclosing multiple parallel loops (PR)**, **parallelizing array reductions (PA)**, **avoiding load imbalance through dynamic scheduling (DS)**, and **eliminate barrier synchronizations using NOWAIT clause (NW)**.

These patterns were identified as key challenge areas for classical optimizing compilers [7], [33], an important question is: **Can LLMs identify these patterns and perform the requisite optimizations?** More detailed explanation of these challenge patterns can be found in prior studies [7].

4 Evaluation: LLMs versus Classical Optimizing Compilers

This section assesses the current capabilities of two Large Language Models, GPT-4.0 and CodeLlama-70B, in comparison with three different automatic compilers, CETUS, PLUTO and ROSE. Section 4.1 outlines the experimental setup, Section 4.2 evaluates the ability of the LLMs to process and correctly optimize code of different sizes. Section 4.3 examines the ability of these LLMs to optimize code sections effectively and also assesses their capacity in handling the challenge patterns introduced in Section 3.2. Lastly, Section 4.4 compares the achieved speedup of all five Automatic Optimization Tools (AOTs).

4.1 Experimental setup

To assess our Automatic Optimization Tools, we use the PCAOT environment described in Section 3.1. Our test suite drew from three distinct benchmarks: The NPB v3.3 [5], the PB suite V4.2 [38], and the PCB benchmark introduced in Section 3.2. We measured the performance of the applications using input Class B for the NPB and the LARGE_DATASET for the PB and PCB suite. Recall that we are focusing on challenge patterns for the optimizing compilers. We have selected specific subroutines from the NPB and the PB suites that contains such cases with a total of 16 experimental sections. For the PCB, we developed 28 use cases, with each one representing one of the six challenge scenarios described in Section 3.2. In total, our testing dataset contains 44 experimental sections. More details about the testing data set can be found on our GitHub repository [25].

We set the following AOT parameters: For the CETUS, PLUTO and ROSE compilers we chose their default options, selecting the

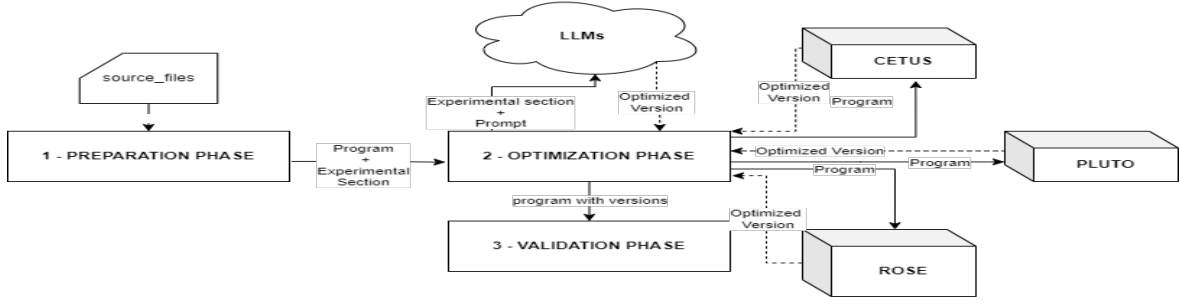


Figure 1: Architecture of the PCAOT Environment.

known, safe optimization behavior. For the LLMs, we chose a Temperature of **0.2** and a Top_p value of **0.1**, increasing deterministic behavior for the code generation tasks.

We measured execution times using 4 cores on a compute node featuring an Intel Xeon Gold 6230 processor configuration in dual sockets. Each processor operates at a base frequency of 2.1 GHz, with a 27.5MB cache, and was supported by up to 1 GB of DDR4 memory. Application codes were compiled using GCC v11.2 with -O3 optimization on CentOS v7.4.1708. Reported values represent the median of three application runs, each utilizing one thread per core.

4.2 Impact of Code Size

Figure 3, shows the failure rate of the Large Language Models in generating programs with increasing code sizes. An unsuccessful case refers to instances where the LLMs failed to produce output that passed validation. The graph displays the number of lines of code per experimental section versus the unsuccessful cases. We sent the sections to GPT4.0 and Codellama-70B, using the three different prompts described in Section 3.1.1, repeating each run three times, with a total of 792 attempts (44 experimental sections x 3 times each x 3 prompts x 2 LLMs = 792 attempts). Details about the experimental sections used in this study can be found on our GitHub repository [25].

These results picture the LLMs' capability to handle large code sizes. While smaller code sections tend to be transformed more accurately, they are not guaranteed to be 100% correct. As the code size increases, the likelihood of inaccuracies also increases.

Program sections of approximately 20 lines often produce correct output, whereas sections above 20 lines increase the failing percentage (performance will be evaluated later). In between, the LLMs exhibit non-deterministic behavior, with some runs producing useful results while others with the same input fail. GPT4.0 and Codellama-70B return successfully transformed code in 16.75% of the cases. These results corroborate our method of employing LLM capabilities on the basis of individual program sections, such as loops.

4.3 LLM Optimization Capabilities

In this subsection, we evaluate the capacity of Large Language Models (LLMs) to optimize both parallel and non-parallel scenarios. Parallel use cases represent scenarios where the LLMs must identify

opportunities for parallelization and apply the correct optimization strategy. In contrast, non-parallel use cases test the LLMs capability to accurately recognize scenarios where parallelization is not feasible or beneficial. For this purpose, we utilized 94 parallel use cases and 10 non-parallel use cases.

4.3.1 Parallel Use cases: Figure ?? shows the success rates of generating correctly optimized code by the tested LLMs on specific challenge patterns, using the three prompts described in Section 3.1.1. The results are classified into three distinct categories: **Expected Pattern applied, Unexpected Pattern Correctly Applied and Errors** (Compilation and Runtime Errors).

Our primary finding from this information is that LLMs are not yet in a position where they can serve as automatic optimizers. Only in three of the analyzed optimization patterns, both LLMs produce partly correct results. A key demand on automatic optimizers is that they be 100% correct; tools that require software engineers to engage in possibly lengthy debug phases after use are not acceptable. The partial successes in three of the patterns (Parallelizing Outermost Loop, Parallelization with Function Calls and Array Reduction) indicate that LLMs may be making progress toward the goal of becoming useful AOTs. These three patterns represent some of the most important parallelization capabilities and likely have many code examples serving as LLM training data, which may explain the partial success. The Parallel-Region and NOWAIT patterns tend to involve large code regions, which challenge LLMs, as discussed in Section 4.2. The two LLMs completely fail on Dynamic Scheduling patterns. We attribute this behavior to the more complex code exhibited by such test cases. Dynamic scheduling tends to be needed in loops that involve irregular code and data structures, which are more difficult to comprehend.

The results show a substantial difference between the three prompting strategies, with DIP being better among the three different approaches in most of the cases. While other researchers have reported more success with CoT prompting, this method performed essentially similarly to DIP.

4.3.2 Non-parallel Use cases. Figure 5 illustrates the correctness of the non-parallel use cases. For this analysis, we evaluate loops with data dependencies (DP) and loops with non-side effect free function calls (FC).

- **Non-Parallel-Loop-DP:** This category represents code sections with data dependencies that prevent safe parallelization.

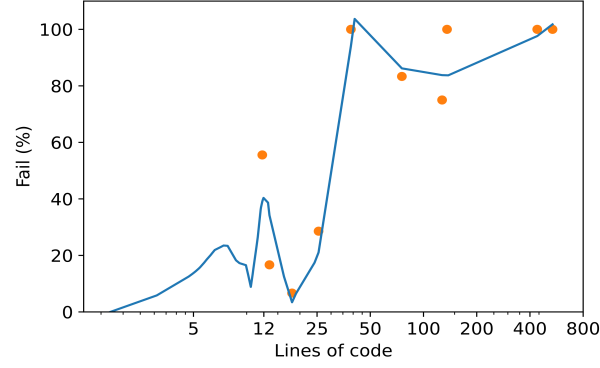


Figure 3: Average failure rate of the Large Language Models across benchmarks when generating programs with increasing code sizes.

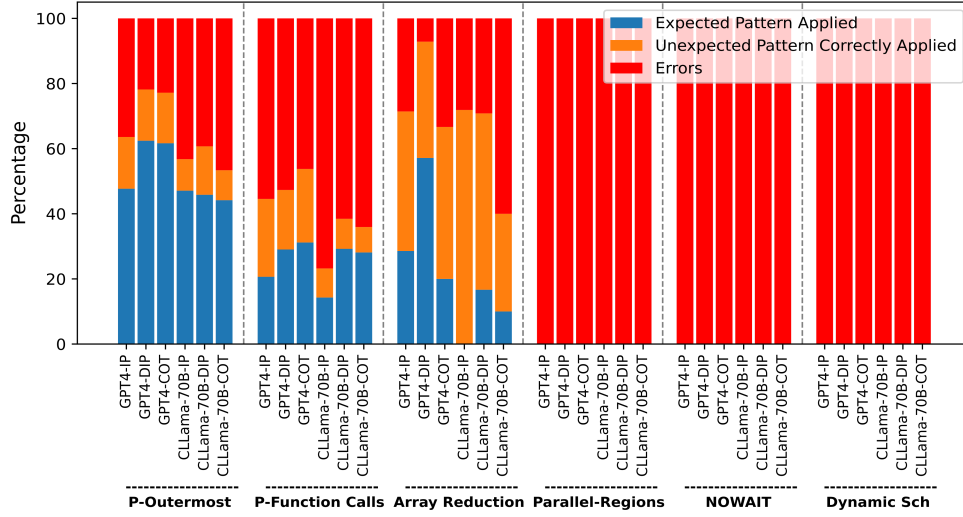


Figure 4: Correctness Evaluation for the challenge patterns: Parallelization at the Outermost level (P-Outermost), Parallelization for function calls (P-Function Calls), Array Reduction, Parallel-Regions, NOWAIT, and Dynamic Scheduling (Dynamic Sch).

- **Non_Parallel_Loop_FC:** This category represents code sections containing function calls that are not side-effect-free, thereby making data dependence analysis required for safe parallelization more complex.

These categories help differentiate the specific reasons why loops in the negative use cases cannot be parallelized, assessing the LLMs' ability to correctly recognize non-parallelizable scenarios. The results show that the DIP strategy performed marginally better than the CoT approach in recognizing non-parallel use cases, a similar result was found for the parallel use cases show in Section 4.3.1. However, both LLMs failed entirely to identify non-parallel code when using IP. We attribute this to the lack of specificity in the IP prompt, as explained in Section 3.1.1.

4.4 Speedup Evaluation by Pattern

Figure 6 shows the maximum average speedups of the correctly optimized codes for the parallel scenarios from the two chosen LLMs. These scenarios include GPT4.0 and CodeLlama-70B across our testing Dataset described in [25] with the three different prompts described in Section 3.1.1 for three specific patterns: Parallelization at the Outermost level, Array Reduction and Parallelization for function calls.

Our results demonstrate the efficacy of CodeLlama-70B, which achieves speedups of up to **1.95x** and **1.79x** over the original program. Additionally, the DIP prompt is more efficient for this particular AOT. We attribute this behavior to the type of model that is being used. CodeLlama-70B is fine-tuned to follow instructions more closely. [24].

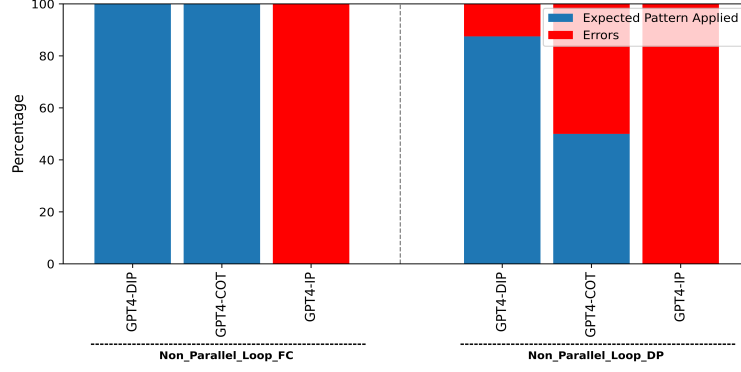


Figure 5: Correctness Evaluation for the challenge patterns: Parallelization at the Outermost level and Parallelization for function calls in non-parallel Scenarios. The expected behavior in this case is that the LLM recognizes the code as non parallelizable.

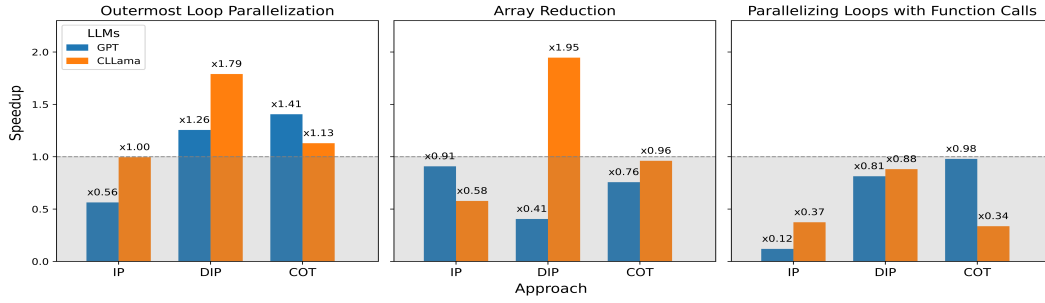


Figure 6: Speedup Evaluation two LLMs (AOTs) across three challenging patterns

4.5 Speedup Evaluation for each AOT

Figure 7 shows the maximum average speed-up of the correctly Parallel optimized codes obtained from the five distinct AOTs over the serial and the Hand-optimized versions.

Our results show CodeLlama-70B as the superior optimizer among Language Model-based (LLM) solutions, capable of achieving speedups of up to **1.75x** compared to the original program.

Conversely, we can observe a noticeable difference between all AOTs and PLUTO in terms of the speedup achieved within the PB suite, where PLUTO outperformed all AOTs with a remarkable **7x** speedup. This superior performance is attributed to PLUTO’s specialization on the Polyhedral model [11]. The specialization may also explain why PLUTO was unable to process the other benchmark suites.

Overall, CETUS emerged as the best among the optimizing compilers for processing a variety of programs within these suites, achieving a maximum speedup of **1.67x**. The ROSE optimizer achieved a speedup across all benchmarks of **1.17x**, remaining below the other AOTs in terms of efficacy.

While GPT4 has demonstrated success across various tasks, it exhibited the lowest performance for code optimization across all the benchmarks and among the LLMs. It achieved a maximum speedup of **1.34x**, falling short of CodeLlama-70B.

5 Related work

Recent studies have evaluated the performance of LLMs in specific High-Performance Computing (HPC) optimization tasks, such as OpenMP pragma prediction [29] highlighting the application of LLMs in parallel computing optimizations. Additionally, approaches to code completion [36] and issue fixing [36] show the breadth of LLM applications in software development and maintenance.

Another significant contribution in this domain is the novel paradigm introduced by researchers using LLMs with compiler feedback to optimize the code size of LLVM assembly. Their model takes unoptimized LLVM Intermediate Representation (IR) as input and produces optimized IR, optimization passes, and instruction counts for both unoptimized and optimized IRs [15].

Recent studies have evaluated the ability of LLMs to generate short programs from natural language descriptions, demonstrating the potential of these models in code generation tasks [3]. However, despite of these advancements, the automatic validation of both the correctness and performance of optimizations remains underexplored.

The primary objective of PCAOT is to automatically validate the correctness and performance of optimizations generated by Automatic Optimization Tools (AOTs). Our work aims to fill this gap by providing a comprehensive framework for evaluating and

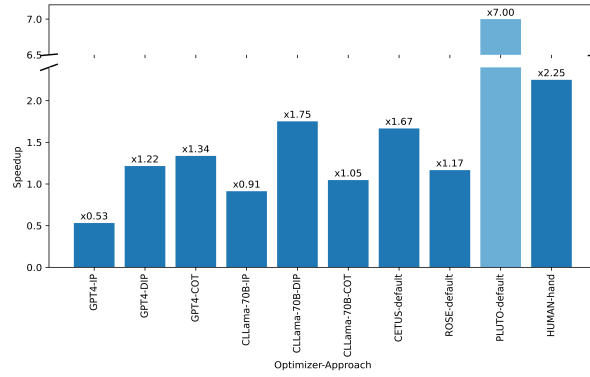


Figure 7: AOTs - Speedup Evaluation of each AOT using a specific configuration: Simple Instruction, Instruction prompting, and Chain-of-Thought for LLMs and the default configuration for the optimizing compilers.

validating the outputs of AOTs, ensuring that the generated optimizations meet the desired standards of correctness. To the best of our knowledge, no other research has directly addressed this specific objective.

Last but not least, several benchmarks have been proposed to evaluate the capacity of AOTs on their abilities and limitations in optimizing code for maximum efficiency [5], [38], [9]. We proposed a novel Parallel Computing Challenge Benchmark suite (PCB) V1.0 containing 28 use cases, each representing one of the six challenge patterns for evaluating AOT code optimization capabilities. To the best of our knowledge, no other benchmark have been designed to directly address this specific goal.

6 Conclusions

This paper addresses an existential question of the compiler community: *"Can AI-driven models revolutionize the way we approach code optimization?"* To this end we evaluated five different Automatic Optimization Tools (AOT - three classical compilers and two LLMs), assessing their respective abilities and limitations in optimizing code for maximum efficiency. To support this evaluation, we introduced a novel mechanism called PCAOT, which enables the automatic validation of the correctness and performance of code generated by AOTs. While we have applied the mechanism to verify LLM-generated code, it also enables unsafe compiler optimization to be tested. Additionally, we have presented the Parallel Computing Challenge Benchmark suite (PCB) v1.0, comprising 28 parallel and non-parallel use cases for evaluating the capabilities of AOTs. To the best of our knowledge, this is the first such benchmark.

Our results show that LLMs only succeed in correctly optimizing very small programs of approximately 20 lines. Given that real-world software often contains thousands of lines of code, these models are not yet ready to serve as automatic optimizers.

In terms of performance, the chosen LLMs can successfully identify three out of the six different challenge patterns, including Array Reduction, Parallelizing Outermost Loops, and Loops Containing Function Calls, with a success rate higher than 30% in most cases. However, they completely fail on other patterns, including eliminating barrier synchronization using the NOWAIT clause, parallel

regions enclosing multiple parallel loops, and avoiding load imbalance through dynamic scheduling.

We found substantial differences between three prompting strategies we employed for both the parallel and non-parallel scenarios, with "Detailed Instruction Prompting" (DIP) being better among the three different approaches in most of the cases. Additionally, "Chain-of-Thought" (CoT) prompting performed essentially similarly to DIP. We attribute this behavior to the high-level compiler knowledge strategies expressed in these two prompts.

In terms of speedup, the best compiler and the best LLM performed within 10%. CodeLlama, which is specifically optimized for code synthesis and infilling/completion tasks [14], was expected to demonstrate strong performance in this context. However, ChatGPT contains a larger number of AI model parameters, which may make up for the fact that it is a general model.

Finally, addressing the main question asked in the introduction, we find that, with their current capabilities, the LLMs cannot yet suitably replace classical compiler optimization technology. Even though the potential exists, future LLMs will need significant improvements in terms of both performance and correctness. Future work will explore additional factors, such as the energy consumption required to achieve performance gains over conventional optimizing compilers. We also anticipate that our results will remain consistent across different platforms.

Acknowledgments

This work was supported in part by the University of Delaware and by the National Science Foundation under awards 2112606, 2125703, and 1931339.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Nicholas Asher, Swarnadeep Bhar, Akshay Chaturvedi, Julie Hunter, and Soumya Paul. 2023. Limits for learning with language models. *arXiv preprint arXiv:2306.12213* (2023).
- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

- [4] Hansang Bae, Dheya Mustafa, Jae-Woo Lee, Aurangzeb, Hao Lin, Chirag Dave, Rudolf Eigenmann, and Samuel P Midkiff. 2013. The cetus source-to-source compiler infrastructure: overview and evaluation. *International Journal of Parallel Programming* 41 (2013), 753–767.
- [5] David H Bailey, Eric Barszcz, John T Barton, David S Browning, Robert L Carter, Leonardo Dagum, Rod A Fatoohi, Paul O Frederickson, Thomas A Lasinski, Rob S Schreiber, et al. 1991. The NAS parallel benchmarks—summary and preliminary results. In *Proceedings of the 1991 ACM/IEEE Conference on Supercomputing*. 158–165.
- [6] Parinaz Barakhshan and Rudolf Eigenmann. 2023. CaRV—Accelerating Program Optimization through Capture, Replay, Validate. In *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. 654–662.
- [7] Parinaz Barakhshan and Rudolf Eigenmann. 2023. Learning from Automatically Versus Manually Parallelized NAS Benchmarks. In *International Workshop on Languages and Compilers for Parallel Computing*. Springer, 29–46. https://link.springer.com/chapter/10.1007/978-3-031-31445-2_3.
- [8] Akshay Bhosale, Parinaz Barakhshan, Miguel Romero Rosas, and Rudolf Eigenmann. 2022. Automatic and interactive program parallelization using the Cetus source to source compiler infrastructure v2. 0. *Electronics* 11, 5 (2022), 809.
- [9] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, et al. 2022. Gpt-neox-20b: An open-source autoregressive language model. *arXiv preprint arXiv:2204.06745* (2022).
- [10] Uday Bondhugula, Muthu Baskaran, Sriram Krishnamoorthy, J. Ramanujam, A. Rountev, and P. Sadayappan. 2008. Automatic Transformations for Communication-Minimized Parallelization and Locality Optimization in the Polyhedral Model. In *International Conference on Compiler Construction (ETAPS CC)*.
- [11] Uday Bondhugula, Muthu Baskaran, Sriram Krishnamoorthy, J. Ramanujam, Atanas Rountev, and P. Sadayappan. 2008. Automatic Transformations for Communication-Minimized Parallelization and Locality Optimization in the Polyhedral Model. In *Compiler Construction*, Laurie Hendren (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 132–146.
- [12] Le Chen, Pei-Hung Lin, Tristan Vanderbruggen, Chunhua Liao, Murali Emani, and Bronis de Supinski. 2023. Lm4hpc: Towards effective language model application in high-performance computing. In *International Workshop on OpenMP*. Springer, 18–33.
- [13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [14] Wenhan Xiong Grattafiori, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).
- [15] Dejan Grubisic, Chris Cummins, Volker Seeker, and Hugh Leather. 2024. Compiler generated feedback for Large Language Models. *arXiv preprint arXiv:2403.14714* (2024).
- [16] Prakhhar Gupta, Cathy Jiao, Yi-Ting Yeh, Shikib Mehri, Maxine Eskenazi, and Jeffrey P Bigham. 2022. InstructDial: Improving zero and few-shot generalization in dialogue through instruction tuning. *arXiv preprint arXiv:2205.12673* (2022).
- [17] Tal Kadosh, Nadav Schneider, Niranjana Hasabnis, Timothy Mattson, Yuval Pinter, and Gal Oren. 2023. Advising openmp parallelization via a graph-based approach with transformers. In *International Workshop on OpenMP*. Springer, 3–17.
- [18] Charles E. Leiserson, Neil C. Thompson, Joel S. Emer, Bradley C. Kuszmaul, Butler W. Lampson, Daniel Sanchez, and Tao B. Schardl. 2020. There's plenty of room at the Top: What will drive computer performance after Moore's law? *Science* 368, 6495 (2020), eaam9744. doi:10.1126/science.aam9744 [arXiv:https://www.science.org/doi/pdf/10.1126/science.aam9744](https://www.science.org/doi/pdf/10.1126/science.aam9744)
- [19] Fengqian Li, Feilong Tang, and Yao Shen. 2014. Feature mining for machine learning based compilation optimization. In *2014 Eighth International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing*. IEEE, 207–214.
- [20] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
- [21] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [22] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *arXiv:2305.01210 [cs.SE]*
- [23] META. [n. d.]. *CodeLlama B-70*. <https://huggingface.co/chat/>
- [24] META. [n. d.]. *CodeLlama B-70 tokens*. <https://huggingface.co/codellama/CodeLlama-70b-Instruct-hf>
- [25] Miguel Torres Miguel Rosas. [n. d.]. *Testing Data Set*. <https://github.com/mgltorsa/AOT-Testing-Suite>
- [26] Miguel Rosas, Miguel Torres. [n. d.]. *Parallel Computing Challenge Benchmark suite (PCB) v1.0*. <https://github.com/mgltorsa/Parallel-Computing-Challenge-Pattern-Benchmark-Suite.git>
- [27] Swaroop Mishra, Daniel Khashabi, Chitta Baral, Yejin Choi, and Hannaneh Hajishirzi. 2021. Reframing Instructional Prompts to GPTk's Language. *arXiv preprint arXiv:2109.07830* (2021).
- [28] Antoine Monsifrot, François Bodin, and Rene Quiniou. 2002. A machine learning approach to automatic production of compiler heuristics. In *Artificial Intelligence: Methodology, Systems, and Applications: 10th International Conference, AIMSA 2002 Varna, Bulgaria, September 4–6, 2002 Proceedings* 10. Springer, 41–50.
- [29] Daniel Nichols, Aniruddha Marathe, Harshitha Menon, Todd Gamblin, and Abhinav Bhatel. 2023. Modeling parallel programs using large language models. *arXiv preprint arXiv:2306.17281* (2023).
- [30] OpenAI. [n. d.]. *ChatGPT*. <https://chat.openai.com/>
- [31] OpenAI. [n. d.]. *GPT4*. <https://platform.openai.com/docs/models/gpt-4-turbo-and-gpt-4>
- [32] Soratouch Pornmaneerattanatri, Keichi Takahashi, Yutaro Kashiwa, Kohei Ichikawa, and Hajimu Iida. 2024. Parallelizable Loop Detection using Pre-trained Transformer Models for Code Understanding. In *Parallel and Distributed Computing, Applications and Technologies*, Ji Su Park, Hiroyuki Takizawa, Hong Shen, and James J. Park (Eds.). Springer Nature Singapore, Singapore, 32–42.
- [33] S Prema, R Jehadeesan, and BK Panigrahi. 2017. Identifying pitfalls in automatic parallelization of NAS parallel benchmarks. In *2017 National Conference on Parallel Computing Technologies (PARCOMPTECH)*. IEEE, 1–6.
- [34] Dan Quinlan and Chunhua Liao. 2011. The ROSE source-to-source compiler infrastructure. In *Cetus users and compiler infrastructure workshop, in conjunction with PACT*, Vol. 2011. Citeseer, 1.
- [35] Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob Gardner, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. 2023. Learning performance-improving code edits. *arXiv preprint arXiv:2302.07867* (2023).
- [36] Pedro Valero-Lara, Alexis Huante, Mustafa Al Lail, William F Godoy, Keita Teranishi, Prasanna Balaprakash, and Jeffrey S Vetter. 2023. Comparing Llama-2 and GPT-3 LLMs for HPC kernels generation. *arXiv preprint arXiv:2309.07103* (2023).
- [37] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 24824–24837. https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf
- [38] T. Yuki and L.N. Pouchet. [n. d.]. *PolyBenchC-4.2.1*. Accessed on 2021-01-05.
- [39] Min Zhao, Bruce R Childers, and Mary Lou Soffa. 2005. A model-based framework: an approach for profit-driven optimization. In *International Symposium on Code Generation and Optimization*. IEEE, 317–327.